Refined Proposal

1 Introduction

From the literature review we can draw the following limits for current belief based approaches:

- while they use VAE to estimate the state distribution, they do not infer the uncertainty of the distribution. They sample from the belief distribution but they do not use sampling to estimate uncertainty.
- this uncertainty is not used in the policies
- even most of belief based methods still condition the belief distribution on the history and hence use RNNs while it is not necessary in theory
- the decoders used infer possible trajectories given the latent state. Often, the number of possible trajectories can be infinite, which puts a lot of burden on the decoder. It may be better to have a VAE where the encoder takes (observation, previous belief state) and outputs a new belief sate then the decoder reconstructs (observation, previous belief state).
- two identical agents with the same information should compute the same information; this will not be the case if the computation involves sampling. Ideally we should avoid sampling from the distribution b_t. We should find a way to input the whole distribution into an agent directly.

Complement intrinsic rewards for POMDPs with SLAC: IR does not train by reconstructing the observation but they project the observation into an embedding space and train by comparing this embedding to a reconstruction that is also in the embedding.

Given those limits we have two objectives:

- **principal objective:** Use the uncertainty on the belief distribution to guide the policy towards better states
- **Secondary objective:** get rid of RNNs

For the principal objective, we decided that we want to use entropy in Distributional returns as a proxy for the belief state uncertainty.

2 Proposal

To adapt SLAC so that it does not behave like Q-MDP, we need to address the key limitation of Q-MDP: its assumption that uncertainty disappears after one action. This means SLAC needs to explicitly reason about long-term uncertainty instead of just maximizing the expected Q-value at the current step.

Q-MDP never chooses actions just to reduce uncertainty (e.g., gathering information).

Solution 1: Introduce reward bonuses for reducing uncertainty in the belief state. Define an information gain reward (like in Bayes-Adaptive RL):

$$r_{\text{info}} = H[q(z_t)] - H[q(z_{t+1})]$$

where $H[q(z)]$ is the entropy of the belief distribution.

Solution 2: Use a belief-space policy rather than a myopic latent-state policy. Instead of training the policy as $\pi(a_t|z_t)$, train it as $\pi(a_t|q(z_t))$, explicitly conditioning on the full belief.

Solution 3: Modify the latent dynamics model to predict not just the next state but also belief evolution. Instead of using a standard latent transition model $p(z_{t+1}|z_t, a_t)$, use a belief transition model:

$$q(z_{t+1}) = \int p(z_{t+1}|z_t, a_t) q(z_t) dz_t$$

models how the entire belief distribution shifts over time, not just the most likely state.

Going with solution 1:

$$r_{\text{info}} = H[q(z_t)] - H[q(z_{t+1})]$$

It supposes that we compute the entropy of the latent space distribution q, which is hard.

Suppose that I have a model that can compute the distribution of the return for any belief, action pair. This distribution is 1D and it is easy to compute its entropy. Suppose that for a given pair, the entropy is high, this means that the uncertainty over future returns is high. This can have two causes: either the environment is stochastic or the belief is actually uncertain.

So I can sample sample different beliefs compute their return distribution and get the uncertainty associated with each belief.

This assumption holds well in partially observable settings, where:

- If belief uncertainty is high, then the return depends on many possible hidden states, leading to high entropy in the probability of returns.

- If the belief is certain but the environment is stochastic, return entropy will still be high, but this is an unavoidable factor.

So, return entropy can serve as an **upper bound on belief entropy**, meaning it's useful for active information gathering.

Question: Is it possible to combine these uncertainties (entropies over 1D distributions) to get the entropy over the entire belief distribution?

Approach 1: Let's say we sample N beliefs $q(z_t^{(i)})$ and compute their respective return entropies:

$$H[p(R|q(z_t^{(i)}), a_t)]$$

for each sample i. A straightforward approach is to average across sampled beliefs:

$$H[q(z_t)] \approx \frac{1}{N} \sum_{i=1}^{N} H[p(R|q(z_t^{(i)}), a_t)]$$

This approach is easy but it ignores how belief entropy actually contributes to return entropy In other terms, **is it possible to separate the entropy caused by the environment stochasticity and the one caused by the uncertainty over belief?**

Approach 2: Mutual Information

A more principled approach is to measure the mutual information between beliefs and returns:

$$I(Z; R) = H[R] - \mathbb{E}_{q(z_t)}[H[p(R|z_t, a_t)]]$$

where:

- $H[R]$ is the entropy of the overall return distribution.
- $H[p(R|z_t, a_t)]$ is the conditional entropy of returns given a specific latent state.

This measures how much knowing the latent state reduces return uncertainty. If we assume that high return entropy is caused mostly by belief uncertainty (not environment randomness), then mutual information $I(Z; R)$ is a good proxy for belief entropy.

How to compute mutual information?

For $\mathbb{E}_{q(z_t)}[H[p(R|z_t, a_t)]]$ we can:

1. Sample $z_t^{(i)} \sim q(z_t)$
2. for each $z_t^{(i)}$ generate a return distribution $R^{(i)} \sim p(R|z_t^{(i)})$
3. compute their entropies and average them

For $H[R]$, we can:

1. Sample $z_t^{(i)} \sim q(z_t)$
2. for each $z_t^{(i)}$ generate a return distribution $R^{(i)} \sim p(R|z_t^{(i)})$
3. Compute the "mean" distribution
4. Compute its entropy

How to integrate that into SLAC

We recall the ELBO from SLAC including the the optimality criteria:

$$\begin{aligned}\text{ELBO} = \mathbb{E}_{(z_{1:T}, a_{\tau+1:T})\sim q}[\sum_{t=0}^{\tau} (\log p(x_{t+1}|z_{t+1}) - \text{KL}(q(z_{t+1}|x_{t+1}, z_t, a_t)||p(z_{t+1}|z_t, a_t))) \\ + \sum_{t=\tau+1}^{T} (r(z_t, a_t) + \log p(a_t) - \log \pi(a_t|x_{1:t}, a_{1:t-1}))].\end{aligned} \tag{1}$$

In the second part we have: The reward $r(z_t, a_t) = \log p(O_t|z_t, a_t)$, representing the agent's objective. using this reward to represent $\log p(O_t|z_t, a_t)$ is just a "design" choice. We can totally replace it by an uncertainty aware reward. If we define:

$$r_{\text{total}}(s_t, a_t) = r_{\text{env}}(s_t, a_t) + \lambda \cdot r_{\text{info}}(z_t, a_t)$$

But now we can use:

$$r_{\text{info}} \approx I(Z; R_{t+1}) - I(Z; R_t)$$

This means r_{info} can be interpreted as the change in how much belief uncertainty influences return uncertainty over time.

Intuition: If an action reduces belief uncertainty and makes returns more predictable, then $I(Z; R)$ should increase over time meaning the agent gains useful information about the latent state.

How to integrate this new reward to the SLAC objective?

In the SLAC ELBO, the authors replace $+ \sum_{t=\tau+1}^{T}(r(z_t, a_t)$ by $\mathbb{E}_{a_{\tau+1}\sim\pi(\cdot|x_{1:\tau+1}, a_{1:\tau})}[Q(z_{\tau+1}, a_{\tau+1})]$.

Unfortunately we cannot replace the sum of our new reward by a Q-value. The Q-function must now learn to approximate not just the expected sum of environmental rewards but also the additional information gain term. This means that if the intrinsic reward r_{info} depends on the belief over latent states, then $Q(z, a)$ should ideally capture how actions influence the evolution of belief entropy, which may require additional training signals or modifications to the critic.

There is also an issue due to **Temporal correlations:** Unlike environmental rewards, which depend mostly on immediate transitions, r_{info} depends on how uncertainty evolves over multiple steps. If this long-term dependency is not well captured, then $Q(z, a)$ may underestimate information gain rewards.

If r_{info} changes dynamically as belief updates, the Q-value function might become harder to estimate, requiring techniques like reward normalization or a separate value function for intrinsic rewards.

To summarize: $r_{\text{info}} \approx I(Z; R_{t+1}) - I(Z; R_t)$ depends on $q(z_t)$ which is **non markovian**, it depends on how uncertainty changes over time, not just the current state.

We will therefore decouple r_{env} and r_{info} and we will use two different NN estimators as described in the plan below:

- The agent is in a belief state z_{t-1} makes an observation o_t
- this observation as well as the last belief state are input to model q that can output samples for the new belief state z_t^i.
- given these new belief samples and the current observation, another model Z predicts a return distribution $Z_t^i(a)$ for each belief sample and action.
- A third model Z' gets z_t^i, computes return distributions $Z_t^{'(i)}(a)$. The average of their entropies and the entropy of the "mean" distribution are used to compute $I(z_t, a_t)$.
- then we need to compute $I(z_{t+1})$, so we need a model p that can learn $p(z_{t+1}|z_t, o_t, a_t)$. Using the sampled $\hat{z}_{t+1}^i$ and Z', we can compute $I(\hat{z}_{t+1})$.
- Using $I(z_t, a_t)$ and $I(\hat{z}_{t+1})$, we can compute r_{info} for each a_t.
- We then use our return distribution model Z to compute for each a_t, $Q(z_t, a_t)$ and the model Z' to compute Q'
- Finally we can choose the action with the best $Q_{info} = Q + \lambda Q'$

So all in all we have 4 models: q, p, Z and Z'

The question is how to train them. Of course we will use the same procedure as SLACK. Let's recall SLAC's ELBO and reformulate it as the desired model:

$$\begin{aligned}\text{ELBO} = \mathbb{E}_{(z_{1:T}, a_{\tau+1:T})\sim q}[\sum_{t=0}^{\tau} (\log p(x_{t+1}|z_{t+1}) - \text{KL}(q(z_{t+1}|x_{t+1}, z_t, a_t)||p(z_{t+1}|z_t, a_t))) \\ + \sum_{t=\tau+1}^{T} (r(z_t, a_t) + r_{info}(z_t, a_t, z_{t+1}) + \log p(a_t) - \log \pi(a_t|z_t))].\end{aligned} \tag{2}$$

- We can first observe that there is $q(z_{t+1}|x_{t+1}, z_t, a_t)$ that learns the new belief given the mast one and the observation.
- We can also notice that there is already $p(z_{t+1}|z_t, a_t)$ that learns the environment dynamics.
- Q and Q' are trained using r and classical Q-update

Let's focus on the ELBO's second part and expand r into the components r_{env} and r_{info}

$$= \mathbb{E}_{z_{\tau+1}\sim q(\cdot|x_{\tau+1},z_\tau,a_\tau)} \left[\mathbb{E}_{a_{\tau+1}\sim\pi(\cdot|x_{1:\tau+1},a_{1:\tau})} \left[Q_{info}(z_{\tau+1}, a_{\tau+1}) - \log \pi(a_{\tau+1}|x_{1:\tau+1}, a_{1:\tau})\right]\right]$$

There are two issues here:

- we need to compute $\log \pi$, which is not doable with the Q learning. We need to use SAC too to have an actor on which we can compute the entropy.
- In our ideal formulation the actor is conditioned on the belief state rather than the history of observations. But as we sample different belief states using q, which belief state should we input in π?

Let's derive the actor objective as in the SLAC paper. I believe we can keep this part as it is (modulo the need to discuss the actor's input and taht we use Q_{info} instead):

$$\begin{aligned}
&\mathbb{E}_{z_{\tau+1}\sim q(\cdot|x_{\tau+1},z_\tau,a_\tau)} \left[\mathbb{E}_{a_{\tau+1}\sim\pi(\cdot|x_{1:\tau+1},a_{1:\tau})} \left[Q(z_{\tau+1}, a_{\tau+1}) - \log \pi(a_{\tau+1}|x_{1:\tau+1}, a_{1:\tau})\right]\right] \\
&\quad = \mathbb{E}_{a_{\tau+1}\sim\pi(\cdot|x_{1:\tau+1},a_{1:\tau})} \left[\mathbb{E}_{z_{\tau+1}\sim q(\cdot|x_{\tau+1},z_\tau,a_\tau)} \left[Q(z_{\tau+1}, a_{\tau+1})\right] - \log \pi(a_{\tau+1}|x_{1:\tau+1}, a_{1:\tau})\right] \\
&\quad = -D_{\mathrm{KL}} \left(\pi(a_{\tau+1}|x_{1:\tau+1}, a_{1:\tau}) \middle\| \frac{\exp\left(\mathbb{E}_{z_{\tau+1}\sim q}[Q(z_{\tau+1}, a_{\tau+1})]\right)}{\exp\left(\mathbb{E}_{z_{\tau+1}\sim q}[V(z_{\tau+1})]\right)}\right) + \mathbb{E}_{z_{\tau+1}\sim q}\left[V(z_{\tau+1})\right]
\end{aligned} \tag{3}$$

And therefore we can keep the same objective:

$$\mathbb{E}_{a_{t+1}\sim\pi_\phi} \left[\alpha \log \pi_\phi(a_{t+1}|x_{1:t+1}, a_{1:t}) - Q_\theta(z_{t+1}, a_{t+1})\right].$$

In SLAC, the actor is trained by sampling trajectories $z_{1:\tau+1} \sim q_\psi$ (via the variational posterior) and optimizing the policy over the entire sequence. This gives the final objective:

$$\pi_\phi = \mathbb{E}_{z_{1:\tau+1}\sim q_\psi} \left[\mathbb{E}_{a_{\tau+1}\sim\pi_\phi} \left[\alpha \log \pi_\phi(a_{\tau+1}|x_{1:\tau+1}, a_{1:\tau}) - Q_\theta(z_{\tau+1}, a_{\tau+1})\right]\right].$$

Discussion about actor input: The actor takes the history of actions and observations as input instead of the current latent state. There are two ways to see this:

- we can keep it this way. One of the main limit of SLAC is that it does not take uncertainty into account, so giving history of observations rather than latent state was not an issue. In our case, the actor is specifically trained to maximize a criteria that encourages it to reduce uncertainty on the latent state. So maybe it can help the actors perform better actions even without seeing the latent state.
- Input a sample from $q(z_t)$ to the actor while also giving it the uncertainty about this sample $I(Z, R)$.
- the model $q(z_t|x_t, z_{t-1})$ also needs a sample z_{t-1}, maybe we can augment this sample with $I(Z_{t-1})$?

So to close this proposal, let's recap again every needed component and its loss function.

1. the latent state estimator is the same as in SLAC, so it is trained with the following objective:

$$J_M(\psi) = \mathbb{E}_{z_{1:\tau+1}\sim q_\psi}\left[\sum_{t=0}^{T} -\log p_\psi(x_{t+1}|z_{t+1}) + D_{KL}(q_\psi(z_{t+1}|x_{t+1}, z_t, a_t)||p_\psi(z_{t+1}|z_t, a_t))\right]$$

   We therefore need three models for state estimation

2. The critic is trained with the following objective:

$$J_{Q(\theta)} = \mathbb{E}_{z_{1:\tau+1}\sim q_\psi}\left[\frac{1}{2}\left(Q_\theta(z_\tau, a_\tau) - (r_\tau + \gamma V_\theta(z_{\tau+1}))\right)^2\right].$$

   but recall that we have two critics now that are used to compute Q_{info}.

   (a) So we have a first critic that uses the objective above. If we deal with discrete action spaces we may don't have to add the V_θ model.

   (b) for the second critic we have the same objective but with Q' and r_{info}

3. Finally there is the actor objective:

$$J_\pi(\phi) = \mathbb{E}_{z_{1:T+1}\sim q_\psi}\mathbb{E}_{a_{T+1}\sim\pi_\phi}\left[\alpha\log\pi_\phi(a_{T+1}|x_{1:T+1}, a_{1:T}) - Q_\theta^{info}(z_{T+1}, a_{T+1})\right]$$

3 Discrete Policy Objective Adaptation

We recall that the policy objective is:

$$\mathcal{J}(\pi) = -D_{\mathrm{KL}}\left(\pi(a_{\tau+1}|x_{1:\tau+1}, a_{1:\tau})\middle\|\frac{\exp\left(\mathbb{E}_{z_{\tau+1}\sim q}[Q(z_{\tau+1}, a_{\tau+1})]\right)}{\exp\left(\mathbb{E}_{z_{\tau+1}\sim q}[V(z_{\tau+1})]\right)}\right) + \mathbb{E}_{z_{\tau+1}\sim q}\left[V(z_{\tau+1})\right]$$

Let's rewrite it as follows:

$$\mathcal{J}(\pi) = -\mathrm{KL}\left(\pi(a_{\tau+1}|h_{\tau+1}) \middle\| \underbrace{\frac{\exp(\bar{Q}(a_{\tau+1}))}{\exp(\bar{V})}}_{\pi^*(a)}\right) + \bar{V}$$

with:

$$\bar{Q}(a) = \mathbb{E}_{z_{\tau+1}\sim q}[Q(z_{\tau+1}, a)], \bar{V} = \mathbb{E}_{z_{\tau+1}\sim q}[V(z_{\tau+1})]$$

Because actions are discrete, the "energy-based" term in $\frac{\exp(\bar{Q}(a_{\tau+1}))}{\exp(\bar{V})}$ is a proper normalised categorical distribution, since:

$$Z \equiv \sum_a \exp(\bar{Q}(a)) \implies \bar{V} = \log Z, \pi^*(a) = \frac{\exp(\bar{Q}(a))}{Z}$$

For a discrete action set $\mathcal{A}$:

$$\mathrm{KL}(\pi\|\pi^*) = \sum_{a\in\mathcal{A}} \pi(a)(\log\pi(a) - \log\pi^*(a)) = \sum_a \pi(a)\log\pi(a) - \sum_a \pi(a)\bar{Q}(a) + \log Z$$

Therefore:

$$\begin{aligned} J(\pi) &= -\left[\sum_a \pi(a)\log\pi(a) - \sum_a \pi(a)\bar{Q}(a) + \log Z\right] + \log Z \\ &= \sum_a \pi(a)\bar{Q}(a) - \sum_a \pi(a)\log\pi(a) \\ &= \sum_a \pi(a|h_{\tau+1})\left[\bar{Q}(a) - \log\pi(a|h_{\tau+1})\right] \\ &= \mathbb{E}_{a\sim\pi}[\bar{Q}(a) - \log\pi(a)] \end{aligned} \tag{4}$$

This is actually the usual soft-Q objective. Maximising $\mathcal{J}$ over all categorical π yields the Boltzmann policy:

$$\pi^*_\alpha(a|h) = \frac{\exp(\bar{Q}(a)/\alpha)}{\sum_{a'}\exp(\bar{Q}(a')/\alpha)}$$

because the derivative w.r.t. $\pi(a)$ enforces $\pi(a) \propto \exp(\bar{Q}(a))$. If we keep only one Q-network and define the policy as that soft-max, we no longer need a separate actor network; the objective is optimised automatically when minimising the soft Bellman error for Q:

$$\frac{1}{2}\left(Q(z_t, a_t) - \left[r_t + \gamma\alpha\log\sum_a e^{Q(z_{t+1},a)/\alpha}\right]\right)^2$$

Therefore all is needed is a Q network and its target network, trained using the soft bellman update and polyak averaging

4 Distributional SLAC with entropy (V1)

The goal is to exploit information about returns inside a SLAC-style agent with a distributional critic (C51), and turn that into a principled intrinsic reward. The key quantities are:

- **Return entropy:** how uncertain the agent is about future returns.
- **Mutual information** $I(Z; R)$: how much knowing the latent state Z reduces uncertainty about returns R.

The discussion boils down to three main points:

1. Don't over-engineer the architecture.
2. difference of differences objectives are noisy
3. Reuse the existing C51 critic to approximate mutual information, instead of adding a new head.

4.1 The "four-model complexity is a trap"

Right now, we already have all the ingredients:

- A SLAC world model that gives a belief distribution over latent state:

$$q(z_t \mid h_t) \tag{5}$$

  where h_t is the history of observations and actions.

- A distributional critic $Z_\theta(z, a)$ (the C51 head), which outputs a categorical distribution over returns for each latent state–action pair.

From just these two, we can derive:

- The expected return:

$$Q_\theta(z, a) = \mathbb{E}[Z_\theta(z, a)] \tag{6}$$

- The entropy of the return distribution given a particular latent and action:

$$H_\theta(R \mid z, a) \tag{7}$$

By resampling z from the posterior $q(z_t \mid h_t)$ and passing those samples through the C51 head, we can build an empirical mixture distribution over returns. That gives us:

- an approximation to the marginal return entropy $H(R)$,
- the conditional entropy $H(R \mid Z)$,
- and thus the mutual information:

$$I(Z; R) = H(R) - H(R \mid Z). \tag{8}$$

Conclusion: we don't actually need a second distributional head Z' just to get an information-based intrinsic reward. The existing critic + belief over latents is enough.

Let's first test MI-based shaping using the current `q_net`. Only if the signal looks promising and stable does it make sense to introduce extra structure.

4.2 Why "difference of differences" is dangerous

The original idea of using a temporal difference of MI,

$$r_t^{\text{info}} = I_{t+1} - I_t \tag{9}$$

has a nice "information gain" interpretation but is numerically very fragile:

- Each $I_t = I(Z_t; R)$ is already a noisy Monte-Carlo estimate (sampled latents, finite atoms, approximate critic).
- Taking a difference $I_{t+1} - I_t$ gives a difference of noisy scalars.
- Feeding that into bootstrapped Q-learning in a POMDP is a recipe for a high-variance, chattery intrinsic reward.

Therefore:

- Don't start with information gain $I_{t+1} - I_t$.
- Start with a simpler reward of the form:

$$r_t^{\text{info}} = \lambda\, I(Z_t; R) \tag{10}$$

  or a normalized/smoothed version of $I(Z_t; R)$.

4.3 Reusing the existing C51 critic to approximate MI

1. For each (batch, time) step, you have a posterior

$$q(z_t \mid x_{\leq t}, a_{<t}).$$

2. Sample latents:

$$z^{(i)} \sim q(z_t \mid h_t), \quad i = 1, \ldots, N.$$

3. Push through the C51 head:

$$p_i(R \mid a_t) = Z_\theta(z^{(i)}, a_t),$$

   i.e., a categorical distribution over atoms for each latent sample.

4. Approximate the marginal return distribution by averaging over latent samples:
$$p_{\text{marg}}(R \mid a_t) \approx \frac{1}{N} \sum_i p_i(R \mid a_t).$$

5. Compute entropies:
   - Total entropy: $H_{\text{tot}} = H[p_{\text{marg}}]$.
   - Conditional entropy: $H_{\text{cond}} \approx \frac{1}{N} \sum_i H[p_i]$.

6. Estimate mutual information:
$$I(Z_t; R \mid a_t) \approx H_{\text{tot}} - H_{\text{cond}}.$$

This gives a scalar MI estimate for each transition and chosen action. We then use it as an intrinsic reward:

$$r_t^{\text{info}} = I(Z_t; R), \tag{11}$$

and train with:

$$r_t^{\text{total}} = r_t^{\text{env}} + \lambda\, r_t^{\text{info}}. \tag{12}$$

Key advantages:

- No new networks.
- All the complexity is in batching and indexing, not in the architecture.
- Conceptually clean: "if I average over my current uncertainty in z, how much extra uncertainty in returns do I get?"

4.4 How to shape the MI bonus (Option A)

Raw MI is always ≥ 0, but:

- its scale is arbitrary and drifts over training,
- it can easily push the agent into "just seek high MI forever" loops.

So we treat MI more like a centered signal than an absolute reward.

Given a matrix $I[b, t]$ of MI estimates for a batch of sequences (shape $(B, S-1)$):

1. Compute batch statistics:
$$\mu_I = \text{mean}_{b,t}\, I[b, t], \quad \sigma_I = \text{std}_{b,t}\, I[b, t] + \varepsilon.$$

2. Standardize:
$$\hat{I}[b, t] = \frac{I[b, t] - \mu_I}{\sigma_I}.$$
   Now $\hat{I}$ is roughly zero-mean and unit variance within the batch.

3. Clip to avoid outliers:

$$\hat{I}_{\text{clip}}[b,t] = \text{clip}(\hat{I}[b,t], -c, c),$$

with $c \in [2,3]$ (e.g. $c = 3$).

4. Scale by λ:

$$r^{\text{info}}[b,t] = \lambda\, \hat{I}_{\text{clip}}[b,t].$$

5. Inject into the replayed rewards: in the sequence setting, if your TD target uses r_{t+1}, you align r_t^{info} with that index (e.g. set `r_all_aug[:, 1:] = r_all[:, 1:] + r_info`).

Why this is a good default:

- **Zero-mean:** on average, the intrinsic term doesn't systematically inflate or deflate values; it just shifts individual transitions up or down.
- **Avoids "always positive" traps:** the agent is not rewarded just for sitting in high-MI regions forever.
- **Automatically keeps scale under control:** normalization + clipping makes λ much easier to tune.

4.5 Alternative smooth "MI gain" (Option B)

If we still want something closer to "MI gain" but without directly differencing adjacent estimates, we can introduce a running baseline:

1. Maintain an EMA of MI over training:

$$\bar{I}_t = (1-\beta) I_t + \beta\, \bar{I}_{t-1}, \quad \beta \approx 0.99. \tag{13}$$

2. Define:

$$r_t^{\text{info}} = \lambda\big(I_t - \bar{I}_t\big). \tag{14}$$

Interpretation:

- If current MI is higher than typical, you get a positive intrinsic reward.
- If it's lower than typical, you get a negative one.
- It's an "advantage-style" signal, but relative to a long-term baseline rather than per-batch mean.

We would still apply global scaling/clipping as in Option A.

Interpretation: "I'm currently uncertain about which latent state I'm in, and those different possibilities imply very different returns. So getting better information about Z would greatly improve my return prediction."

On the plus side, MI is policy-dependent and will naturally shrink as the agent becomes confident and skilled. That's actually a desirable property: the intrinsic naturally decays as the task is mastered.